

FULL STACK DEVELOPMENT INTERNSHIP
FINAL PROJECT REPORT

Nexus - Investor & Entrepreneur Collaboration
Platform

Investor and Entrepreneur Collaboration Platform

Name	Bushra
DHC ID	DHC-2203
Program	DeveloperHub Corporation - Full Stack Development Internship
Project Type	Full Stack Web Application
Frontend	React, TypeScript, Vite, Tailwind CSS
Backend	ASP.NET Core Web API, .NET 8, Entity Framework Core
Database	SQL Server
Repository	https://github.com/Bushr-Afridi7/Bussiness_Nexus.git
Submission Status	Completed

Date: June 2026

Table of Contents

- 1. Executive Summary
- 2. Project Information
- 3. Project Objectives
- 4. Technology Stack
- 5. System Architecture
- 6. Functional Modules Completed
- 7. Backend API Endpoints
- 8. Database Design
- 9. Frontend Pages and Integration
- 10. Testing and Validation
- 11. Completion Matrix
- 12. Challenges and Solutions
- 13. Repository and Submission Details
- 14. Future Enhancements
- 15. Conclusion

1. Executive Summary

Nexus is a complete full stack web application developed as an internship project for DeveloperHub Corporation. The platform is designed to connect investors and entrepreneurs in one professional digital environment. It provides secure authentication, role based dashboards, investor and entrepreneur profile browsing, meeting scheduling, document handling, chat, notifications, mock payments, and real dashboard statistics.

The project was implemented with a React frontend and an ASP.NET Core Web API backend. SQL Server is used for persistent data storage, and Entity Framework Core handles database operations and migrations. All major modules are connected from frontend to backend and tested through Swagger and browser based testing.

2. Project Information

Project Name	Nexus - Investor & Entrepreneur Collaboration Platform
Student Name	Bushra
DHC ID	DHC-2203
Internship Program	DeveloperHub Corporation - Full Stack Development Internship
Repository Link	https://github.com/Bushr-Afridi7/Bussiness_Nexus.git
Application Category	Investor and Entrepreneur Collaboration Platform
Development Status	Completed and ready for final submission

3. Project Objectives

The main objective of this project is to provide a secure, useful, and practical collaboration platform where investors and entrepreneurs can interact professionally. The system supports account management, communication, meetings, document exchange, transaction simulation, and activity tracking.

- Build a full stack application with separate frontend and backend layers.
- Implement secure login, registration, and JWT protected APIs.
- Allow users to operate as either Investor or Entrepreneur.
- Connect all important UI pages with real backend APIs.
- Store project data in SQL Server using Entity Framework Core.
- Provide a complete workflow for meetings, chat, documents, notifications, payments, and dashboards.

4. Technology Stack

Layer	Technologies / Tools	Purpose
-------	----------------------	---------

Frontend	React, TypeScript, Vite	Build fast and component based user interfaces
Styling	Tailwind CSS	Create responsive and modern UI designs
Routing	React Router DOM	Manage page navigation and dynamic routes
API Client	Axios	Send HTTP requests from frontend to backend
Notifications UI	React Hot Toast	Show success and error messages
Icons	Lucide React	Add professional dashboard and action icons
Backend	ASP.NET Core Web API, .NET 8	Create secure REST APIs
ORM	Entity Framework Core	Database operations and migrations
Database	SQL Server	Store users, meetings, messages, documents, notifications, and payments
Testing	Swagger / OpenAPI	Test backend APIs during development
Version Control	Git and GitHub	Manage source code and submission repository

5. System Architecture

The system follows a clean client server architecture. The frontend sends requests to the backend API, the API validates JWT authentication and business rules, and the backend stores or retrieves data from SQL Server. This separation keeps the project organized and easier to maintain.

Component	Description
Nexus.Client	React frontend containing routes, pages, UI components, API configuration, and authentication context.
Nexus.API	ASP.NET Core backend containing controllers, models, DTOs, data

	context, authentication, and migrations.
SQL Server Database	Stores all persistent project data including users, meetings, documents, messages, notifications, and payments.
Swagger	Provides API testing and documentation for backend endpoints.
GitHub Repository	Contains the final project source code for submission and review.

6. Functional Modules Completed

Module	Completed Work	Status
Authentication and JWT Authorization	Registration, login, JWT token generation, protected APIs, and authenticated user profile access.	Complete
Profile Management	Users can view and update their profile information from the settings page.	Complete
Investor and Entrepreneur Listings	Separate pages for investors and entrepreneurs with profile navigation.	Complete
User Profile Pages	View detailed profiles using dynamic user ID routes.	Complete
Meeting Scheduling	Schedule meetings, view my meetings, accept, reject, and cancel meetings.	Complete
Document Chamber	Upload, list, download, sign, reject, and delete documents.	Complete
Chat and Messages	Send messages, view chats, open conversations, and mark messages as read.	Complete
Notifications	Create, view, mark read, mark all read, and delete notifications.	Complete
Mock Payments and Deals	Create mock payments, complete, fail, refund, and track status.	Complete
Dashboard Real Statistics	Investor and entrepreneur dashboards show real database counts.	Complete

7. Backend API Endpoints

The backend contains multiple REST API controllers. All important APIs are protected with JWT authentication where required.

Authentication APIs

Method	Endpoint
POST	/api/Auth/register
POST	/api/Auth/login
GET	/api/Auth/me

Profile APIs

Method	Endpoint
GET	/api/Profile
PUT	/api/Profile

Users APIs

Method	Endpoint
GET	/api/Users/investors
GET	/api/Users/entrepreneurs
GET	/api/Users/{id}

Meetings APIs

Method	Endpoint
POST	/api/Meetings/schedule
GET	/api/Meetings/my-meetings
PUT	/api/Meetings/{id}/accept
PUT	/api/Meetings/{id}/reject
PUT	/api/Meetings/{id}/cancel

Documents APIs

Method	Endpoint
POST	/api/Documents/upload
GET	/api/Documents/my-documents
GET	/api/Documents/{id}/download
PUT	/api/Documents/{id}/sign
PUT	/api/Documents/{id}/reject
DELETE	/api/Documents/{id}

Messages APIs

Method	Endpoint
POST	/api/Messages/send
GET	/api/Messages/conversation/{userId}
GET	/api/Messages/my-chats
PUT	/api/Messages/{id}/read

Notifications APIs

Method	Endpoint
POST	/api/Notifications/create
GET	/api/Notifications/my-notifications
PUT	/api/Notifications/{id}/read
PUT	/api/Notifications/read-all
DELETE	/api/Notifications/{id}

Payments APIs

Method	Endpoint
--------	----------

POST	/api/Payments/create
GET	/api/Payments/my-payments
PUT	/api/Payments/{id}/complete
PUT	/api/Payments/{id}/fail
PUT	/api/Payments/{id}/refund

Dashboard APIs

Method	Endpoint
GET	/api/Dashboard/stats

8. Database Design

The project uses SQL Server as the database and Entity Framework Core for database access. The database contains separate tables for every major module.

Table	Purpose
Users	Stores registered investor and entrepreneur accounts.
Meetings	Stores scheduled meetings and their status.
Documents	Stores document information, file paths, ownership, receiver, and status.
Messages	Stores chat messages between users.
Notifications	Stores user notifications and read status.
Payments	Stores mock payment and deal transaction records.

9. Frontend Pages and Integration

The frontend includes multiple React pages connected with backend APIs through Axios. The token is stored in local storage and automatically used for protected requests.

Route / Page	Functionality
--------------	---------------

/login	User login page.
/register	User registration page.
/dashboard/investor	Investor dashboard with real backend statistics.
/dashboard/entrepreneur	Entrepreneur dashboard with real backend statistics.
/investors	Displays all investor users.
/entrepreneurs	Displays all entrepreneur users.
/profile/investor/:id	Displays investor profile details.
/profile/entrepreneur/:id	Displays entrepreneur profile details.
/meetings	Displays meeting list and actions.
/messages	Displays chat list.
/chat/:userId	Displays conversation and message sending form.
/documents	Document upload and management page.
/notifications	Notification center.
/deals	Mock payment and deals page.
/settings	Profile update and account settings page.

10. Testing and Validation

The application was tested module by module. Backend APIs were tested through Swagger, and frontend pages were tested in the browser after API integration.

- Registration and login tested successfully.
- JWT protected APIs tested with Swagger authorization.
- Investor and entrepreneur listings tested with real database records.
- Profile update tested from the frontend settings page.
- Meeting scheduling, accept, reject, and cancel flow tested successfully.
- Document upload, download, sign, reject, and delete tested successfully.
- Chat send, conversation view, and chat list tested successfully.

- Notifications read, read all, delete, and refresh tested successfully.
- Mock payments create, complete, fail, and refund tested successfully.
- Dashboard statistics tested with real database counts.

11. Completion Matrix

Requirement Area	Implementation	Completion
Authentication	JWT based register, login, and protected APIs	100% Complete
Role Based Users	Investor and Entrepreneur roles supported	100% Complete
Profiles	Profile view and update completed	100% Complete
Meetings	Schedule, accept, reject, cancel, and list meetings	100% Complete
Documents	Upload, share, download, sign, reject, and delete	100% Complete
Messaging	Chat list, conversation, send message, read status	100% Complete
Notifications	Create, view, read, read all, and delete	100% Complete
Payments	Mock payment creation and status management	100% Complete
Dashboard	Real backend statistics connected to dashboards	100% Complete
Repository	GitHub repository created and linked	Complete

12. Challenges and Solutions

Challenge	Solution
Connecting frontend and backend routes	Axios base URL and protected token headers were configured.
JWT authorization testing	Swagger authorization and browser local storage token testing were used.

Role based workflow	Business rules were added so meetings and payments work between opposite role users.
Database integration	Entity Framework Core migrations were used to create and update SQL Server tables.
GitHub upload issues	Nested Git repositories were removed and the full source code was pushed to the main repository.

13. Repository and Submission Details

The final source code is available in the GitHub repository below. The repository contains both frontend and backend folders.

GitHub Repository: https://github.com/Bushr-Afridi7/Bussiness_Nexus.git

Expected repository structure:

Folder / File	Description
Nexus.Client	React frontend source code.
Nexus.API	ASP.NET Core backend source code.
README.md	Professional project documentation.
.gitignore	Excludes node_modules, build folders, bin, obj, and local Visual Studio files.

14. Future Enhancements

- Add real Stripe or PayPal integration instead of mock payments.
- Add real-time chat using SignalR.
- Add real video call integration or meeting provider API.
- Add email notifications for meetings, messages, and document actions.
- Add admin dashboard for platform monitoring.
- Add charts and visual analytics to dashboards.
- Deploy frontend and backend to production hosting platforms.

15. Conclusion

Nexus has been completed as a professional full stack web application according to the internship task requirements. It includes secure authentication, user role management, profile management, meeting scheduling, document chamber, messaging, notifications, mock payments, and real dashboard statistics. Both frontend and backend are integrated successfully, and the project is ready for final review and submission.

Final Status	Completed
Submitted By	Bushra
DHC ID	DHC-2203
Repository	https://github.com/Bushr-Afridi7/Bussiness_Nexus.git